

TECHNICAL REPORT READING NOTES · 2026

DeepSeek V4

技术报告深度解读

从 V3 / R1 / V3.2 到 V4: 百万 Token 上下文、混合注意力、mHC、Muon、系统工程与后训练范式的完整拆解。

Winston

V1.0 · 2026-05-21

基于 DeepSeek 官方 transparency 页面、V4 technical report、V4 model card、V3.2 / V3 / R1 官方论文整理

| 目录

01	执行摘要：V4 到底解决了什么	03
02	事实口径与阅读地图	05
03	代际演进：V3、R1、V3.2 到 V4	07
04	完整架构：MoE、MTP、mHC 与混合注意力	10
05	CSA 与 HCA：百万上下文的核心算法	13
06	mHC 与 Muon：稳定性、收敛与深层信号传播	18
07	系统工程：训练、推理、KV Cache 与确定性	22
08	预训练：数据、模型配置与训练稳定性	27
09	后训练：专家、GRM、OPD、工具调用与沙箱	31
10	评测对比：能力、效率、成本与边界	37
11	创新点分层、局限与实践判断	43
12	术语表与参考来源	47

执行摘要：V4 到底解决了什么

DeepSeek V4 不是单纯把参数量继续做大，而是把“长上下文效率”重新放到架构中心。官方报告给出的核心命题是：在开放权重模型里，把 **1M Token 上下文** 从演示能力推进到可常规支持的工程能力，同时保留强推理、强代码和强 Agent 表现。

一句话结论

V3 证明了 MLA + DeepSeekMoE + FP8 训练可以把 671B MoE 做得高效；R1 证明了 GRPO 与可验证奖励可以释放推理；V3.2 用 DSA 把长上下文成本压低并强化 Agent；V4 则把这些线索合并为新的系统：**混合 CSA/HCA 注意力 + mHC 残差拓扑 + Muon 优化器 + OPD 后训练**。

从指标看，V4-Pro 为 1.6T 总参数、每 token 激活 49B；V4-Flash 为约 284B 总参数、每 token 激活 13B。两者均支持 1M 上下文。官方报告称在 1M 上下文下，V4-Pro 单 token 推理 FLOPs 约为 V3.2 的 27%，KV Cache 约为 V3.2 的 10%；V4-Flash 则约为 10% FLOPs 与 7% KV Cache。

从方法看，V4 的关键不是某一个技巧，而是“端到端共同设计”：注意力压缩改变了 KV 形态，KV 形态反过来要求新的 cache 管理、上下文并行、确定性 kernel、on-disk KV 复用和后训练 rollout 服务。

读完本文应掌握

1. 为什么 V4 需要同时引入 CSA、HCA、mHC 和 Muon，而不是只改注意力。
2. CSA 与 V3.2 的 DSA 有什么关系，HCA 为什么保留 dense attention。
3. mHC 如何把 residual stream 从一条线扩展为受约束的多通道混合。
4. OPD 如何替代 V3.2 的 mixed RL，解决多专家能力合并的问题。
5. 如何客观看待 V4 对 GPT-5.4、Gemini-3.1、Claude Opus 4.6、Kimi K2.6、GLM-5.1 的比较。

FIGURE 1 V4 的四条技术主线

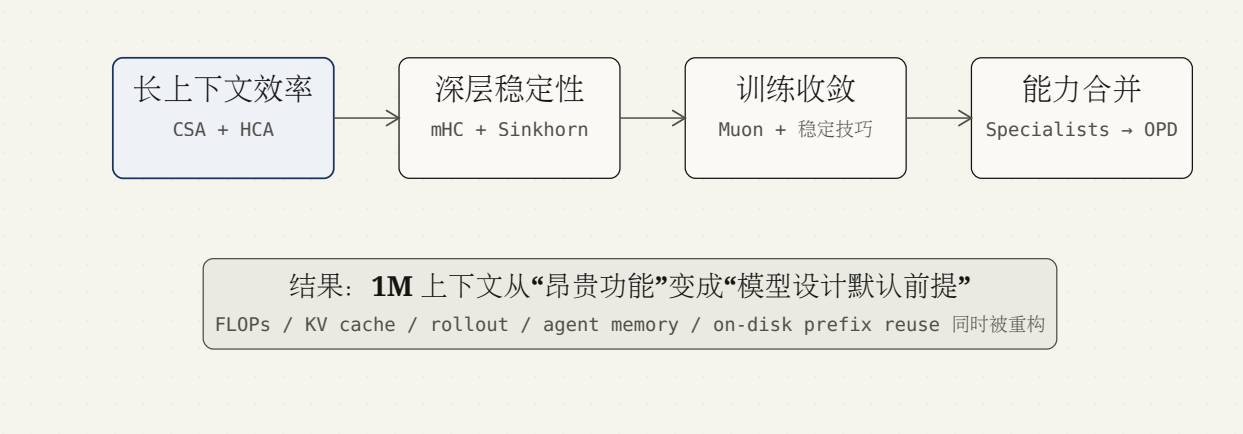


图 1: V4 的主线不是单点算法, 而是架构、优化器、系统和后训练的联合设计。

如何把 V4 记成三句话

第一句话: V4 的首要目标是让百万上下文进入常规工程可用区, 而不是只追求一次性长文本演示。第二句话: 实现这个目标需要先把历史 KV 变成多分辨率记忆, 再围绕这种新 KV 形态重做 cache、并行、量化和 rollout。第三句话: V4 的能力提升来自 pre-training、specialist training 与 OPD 的组合, 因此它不是单一 reasoning 模型, 也不是单一长上下文模型, 而是面向长程任务的综合系统。

读 V4 时最容易误解的一点, 是把 CSA/HCA 当成“注意力加速技巧”。更准确的表述是: CSA/HCA 改变了模型保存、检索和复用历史信息的基本单位; 后面的系统工程几乎都围绕这个新单位展开。

事实口径与阅读地图

这份解读只把可追溯材料作为事实依据，把推断和判断单独标明。尤其是 V4 报告中含有大量内部评测、未来模型命名和自建 benchmark，本文会保留其口径，但不会把它们等同于第三方复现结论。

2.1 当前可核验来源

来源	用途	本文采用口径
DeepSeek 官方透明中心	确认 DeepSeek-V4 页面、发布日期与技术报告入口	页面列出 DeepSeek-V4, Release Date 为 2026-04-24, 报告链接指向 Hugging Face。
DeepSeek_V4.pdf	本文主来源	PDF 共 58 页，标题为 DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence, PDF 元数据显示 2026-05-06 生成。
DeepSeek V4 Model Card	补充模型属性、分发许可、用途与模式	模型卡列 Release date 2026-04-24, 说明 Pro / Flash、文本模态、三种 reasoning modes、1M context、MIT 权重许可。
DeepSeek-V3.2 / V3 / R1 官方论文	代际对比	用于解释 V4 继承了什么、替换了什么、为什么做这些替换。

口径提醒：官方不同材料对 V4-Flash 总参数有轻微差异：technical report 写 284B, model card 写 285B, Hugging Face collection 展示中可见 292B-Base。本文在模型配置表中优先采用 technical report 的 284B，并在讨论中视为约 285B 量级。

2.2 V4 报告的章节结构

V4 官方报告的组织方式很清楚：第 1 章提出问题，指出 vanilla attention 的二次复杂度限制了超长上下文和 test-time scaling；第 2 章讲模型架构，重点是 mHC、CSA、HCA、Muon；第 3 章讲工程基础设施；第 4 章讲预训练；第 5 章讲后训练与评测；第 6 章给出局限和未来方向。读者应把它理解为一份“模型架构 + 训练系统 + 后训练系统”的整体工程报告，而不是一篇只讲 benchmark 的模型卡。

2.3 本文的读法

本文将官方报告重排为学习顺序：先建立问题图景，再讲代际演进，接着逐个拆解模块，最后回到评测与局限。这样做的原因是，直接读官方报告会遇到一个困难：CSA/HCA、mHC、Muon、KV cache 管

理、FP4 QAT、OPD 等模块相互依赖。如果不先理解它们在系统中的位置，单独读公式容易得到“知道每个模块，但不知道为什么它们必须同时出现”的碎片化印象。

读者问题	对应章节	应抓住的主线
V4 相比 V3.2 最大变化是什么？	第 3、4、5 章	从 DSA 长上下文优化升级为 CSA/HCA 的百万上下文架构。
为什么需要 mHC？	第 6 章	让 residual stream 自身成为可扩展、可混合、受约束的表示通道。
Muon 在这里扮演什么角色？	第 6、8 章	优化大部分矩阵参数，使收敛和稳定性服务于更复杂架构。
为什么后训练从 mixed RL 转向 OPD？	第 9 章	先训练多个领域专家，再让统一学生在自己的轨迹上学习多教师分布。
评测能说明什么，不能说明什么？	第 10、11 章	内部评测给出方向性证据，第三方复现仍是边界。

代际演进：V3、R1、V3.2 到 V4

DeepSeek V4 的创新不能孤立理解。V4 的每个关键模块都能在前代模型中找到动机：V3 负责高效 MoE 基座，R1 负责可验证推理范式，V3.2 负责稀疏注意力和 Agent 后训练，V4 则把这些路线推进到百万上下文。

3.1 V3: 高效 MoE 基座的成型

DeepSeek-V3 是后续版本的架构底座。它使用 671B 总参数、每 token 激活 37B 的 MoE，核心结构是 MLA 与 DeepSeekMoE。MLA 通过 latent KV 表示降低推理 cache；DeepSeekMoE 通过细粒度 routed experts 与 shared experts 提供容量；auxiliary-loss-free load balancing 避免为了路由均衡而损害模型质量；MTP 则让每个 token 同时学习预测未来 token，增加训练信号密度。

V3 的另一个重要意义是工程可信度。官方论文报告完整训练约 2.788M H800 GPU hours，14.8T 预训练 token，并声称训练过程中没有不可恢复 loss spike 或 rollback。这为后续“更激进的稀疏和长上下文设计”提供了工程信心。

3.2 R1: 推理能力从监督模仿转向结果驱动

DeepSeek-R1 不是 V4 的直接架构前身，但它改变了 DeepSeek 系列的后训练哲学。R1-Zero 从 V3-Base 出发，跳过常规 SFT，使用 GRPO 和规则奖励直接优化数学、代码、逻辑等可验证任务。关键观察是：只约束最终正确性和输出格式，模型会自发学到更长的推理、自检、反思和策略切换。

R1 的局限同样重要：纯 RL 容易带来可读性差、语言混杂、过度推理和通用任务能力不足。因此 R1 正式版又引入冷启动数据、拒绝采样、SFT、第二阶段 RL 与偏好奖励。V4 的 post-training 并没有简单复制 R1，而是在 V3.2 的基础上转向“多个专家 + OPD 统一合并”。这说明 DeepSeek 已经把推理看成体系化后训练问题，而不是单一 reward trick。

3.3 V3.2: DSA 与 Agent 后训练

V3.2 的核心改动是 DeepSeek Sparse Attention。它从 DeepSeek-V3.1-Terminus 继续训练而来，在 MLA 上加入 lightning indexer 与 top-k token selection，把主注意力复杂度从 $O(L^2)$ 降到 $O(Lk)$ 。这使长上下文成本显著下降，但 V3.2 的上下文长度仍以 128K 为主，Agent workflow 超过窗口后还需要上下文管理策略。

V3.2 在后训练上也迈进一步：官方报告称 post-training 计算预算超过 pre-training 成本的 10%，并构造大规模 Agent 数据，包括 1,827 个 synthesized general-agent environments 与 85,000 个复杂 prompts。这个阶段解决的是“模型能不能在工具交互中维持推理”的问题，而 V4 进一步把这个问题推到百万上下文、保留推理痕迹和更强的沙箱基础设施。

3.4 V4: 从稀疏选择到混合压缩

V4 相比 V3.2 的根本变化在于：不再只是在 token 粒度上稀疏选择历史 KV，而是先把历史 KV 沿序列维压缩，再在压缩后的条目上做稀疏或 dense attention。CSA 继承 DSA 的 lightning indexer 与 top-k 思路，但它面对的是 compressed KV entries；HCA 则使用更高压缩率，保留 dense attention。二者交替使用，使模型同时获得“可选择的精细长程信息”和“极低成本的全局粗粒度信息”。

版本	核心贡献	上下文与效率主线	后训练主线	V4 继承或替换
DeepSeek-V3	MLA、DeepSeekMoE、aux-loss-free、MTP、FP8 训练	经济训练与推理；128K 扩展	SFT + RL，吸收 R1 推理模式	保留 DeepSeekMoE 与 MTP，但注意力不再采用 MLA 主线
DeepSeek-R1	GRPO、规则奖励、长 CoT 自演化	推理 token 扩展带来能力提升	纯 RL 到多阶段对齐	保留 GRPO 作为领域专家 RL 工具，但最终合并转向 OPD
DeepSeek-V3.2	DSA、scalable RL、Agent 数据合成	DSA 把注意力主成本降到 O(Lk)，128K 场景高效	mixed RL 与大规模 agentic task synthesis	CSA 吸收 DSA；V4 用 1M 上下文与 OPD 重构后训练
DeepSeek-V4	CSA/HCA、mHC、Muon、FP4 QAT、OPD、DSec	1M 上下文常规支持；V4-Pro 在 1M 下约 27% V3.2 FLOPs、10% KV	领域专家训练 + full-vocab OPD + long-context RL 基建	形成新的百万上下文开放模型范式

完整架构：MoE、MTP、mHC 与混合注意力

V4 仍是 Transformer-MoE 模型，但每个 Transformer block 的内部连接、注意力形态和优化器假设都变了。最适合的理解方式是：DeepSeekMoE 负责参数容量，CSA/HCA 负责长上下文访问，mHC 负责层间信号传播，MTP 继续作为训练信号增强器。

4.1 两个模型：Pro 与 Flash

项目	DeepSeek-V4-Flash	DeepSeek-V4-Pro
定位	成本效率优先	能力上限优先
总参数	284B，模型卡约 285B	1.6T
每 token 激活参数	13B	49B
Transformer 层数	43	61
Hidden dimension	4096	7168
Query heads	64	128
CSA top-k	512	1024
Routed experts	256，每 token 激活 6 个	384，每 token 激活 6 个
共享专家	1	1
上下文长度	1M	1M
预训练 token	32T	33T

4.2 Block 内部结构

官方图中，V4 的每层 Transformer block 包含 CSA/HCA attention 与 DeepSeekMoE。mHC 并不是替换 attention 或 FFN，而是替换传统残差路径：在每个 block 前后进行 pre-block mixing 与 post-block mixing，使 residual stream 不再只是 $x + F(x)$ ，而是扩展为多个 residual channel 的受约束混合。

MoE 部分大体继承 V3：细粒度 routed experts + shared expert + auxiliary-loss-free load balancing。但 V4 做了三处值得注意的变化：第一，routing affinity activation 从 Sigmoid 改成 Sqrt(Softplus)；第二，移除 routing target nodes 数量约束，并以并行策略保持效率；第三，把最初若干 dense FFN 层替换为 Hash routing 的 MoE 层，V4-Flash 与 V4-Pro 都在前 3 个 MoE 层使用 Hash routing。

FIGURE 2 DeepSeek V4 block-level architecture

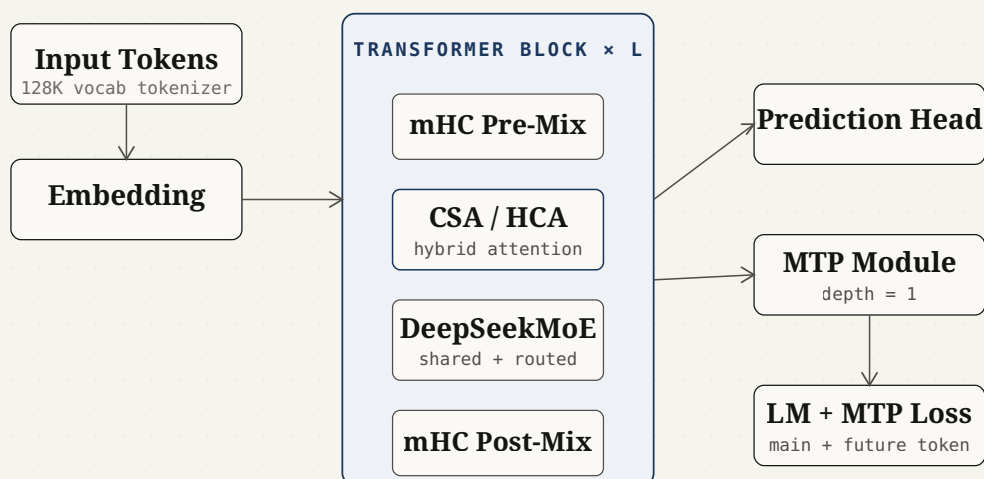


图 2: V4 block 内部的关键变化集中在 residual mixing 与 hybrid attention; MoE 和 MTP 则继承并微调自 V3。

4.3 为什么不是继续沿用 MLA

MLA 的优势在于压缩 KV 表示, 降低常规推理 cache。V3.2 在 MLA 之上加入 DSA, 使 query 只访问 top-k KV latent entries。但当上下文目标从 128K 扩到 1M, 单纯“从 1M 历史 token 中选 top-k”仍面临两个问题: indexer 自身也要看很长历史, KV cache 管理仍以 token-level 或 latent-token-level 为基本单位, 长程粗粒度信息与局部精细信息没有被显式分层。

V4 的 CSA/HCA 相当于把“历史”先变成不同分辨率的压缩条目, 再执行选择或 dense attention。它把序列维从模型结构层面降采样, 而不是只靠 kernel 和 top-k 稀疏计算降成本。这就是 V4 能把 1M context 作为 native 支持目标的原因。

CSA 与 HCA: 百万上下文的核心算法

V4 最重要的算法创新是 hybrid attention: CSA 负责“压缩后再稀疏选择”, HCA 负责“重压缩后保持 dense attention”。二者交替出现, 使模型在 1M 上下文里同时拥有可计算的长程访问、可用的局部细节和可控的 KV cache。

5.1 从 vanilla attention 到 DSA, 再到 CSA/HCA

Vanilla attention 的推理瓶颈来自每个新 token 对全部历史 KV 做 attention, 单 token 成本随上下文长度线性增长, 整段序列训练成本则是二次复杂度。V3.2 的 DSA 通过 lightning indexer 选择 top-k 历史 KV, 把核心 attention 从全量历史降到稀疏历史。但 DSA 的选择对象仍来自历史 token 级别或 MLA latent 条目。

CSA 的思路是两步: 第一, 把每 m 个 token 的 KV 压成一个 compressed KV entry; 第二, 在 compressed entries 上用 lightning indexer 选择 top-k, 再做 core attention。HCA 的思路更激进: 用更大的 m' , 把很长的一段历史压成一个 entry, 然后不做 top-k, 保持 dense attention。可以把 CSA 看成“高分辨率但稀疏的长程记忆”, HCA 看成“低分辨率但连续的全局记忆”。

FIGURE 3 CSA / HCA compression logic

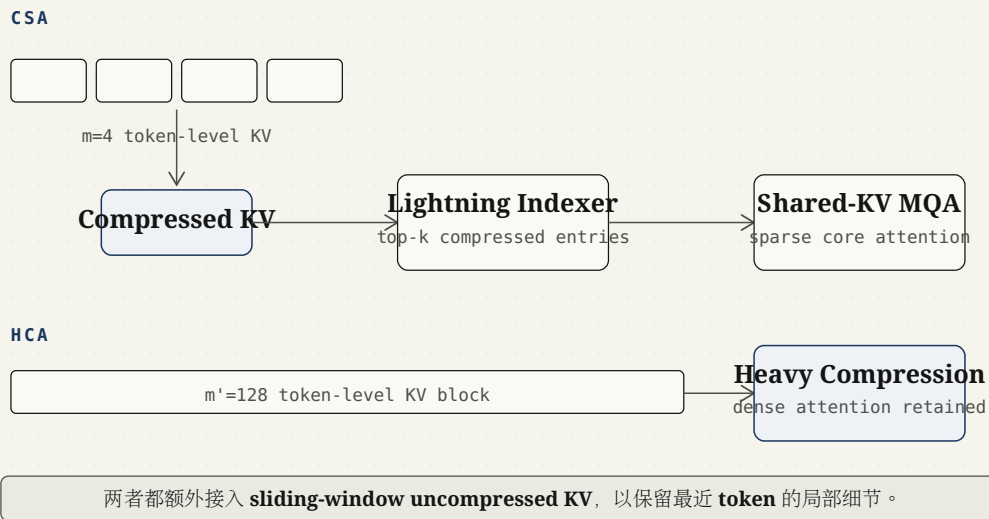


图 3: CSA 使用较小压缩率并稀疏选择; HCA 使用更大压缩率并保留 dense attention。

5.2 CSA 的细节: 压缩、选择、共享 KV MQA

CSA 首先从 hidden states 计算两套 KV entries 与对应压缩权重。每个 compressed entry 不是简单平均, 而是由压缩权重、可学习 positional bias 和 softmax 加权得到。官方公式中, C^a 与 C^b 两条路径存在交叠使用: 一个 compressed entry 来自当前 m 个 C^a 与前一段 m 个 C^b , 这让压缩后的序列长度仍约为原来的 $1/m$, 但每个压缩条目可以吸收跨块边界的信息。

拿到 compressed KV 后，CSA 用 lightning indexer 计算 query 与压缩块的 index score，再选 top-k entries。这里的 indexer query 是低秩生成的，多头 index score 用 ReLU 相似度加权汇总。核心 attention 采用 MQA：compressed KV entry 同时作为 key 与 value，被多个 query heads 共享。这一设计是为了降低 KV 存储和访问成本。

Grouped output projection 解决了另一个计算问题：V4 的 head dimension 与 head 数较大，直接把所有 attention outputs 一次性投回 hidden size 会很重。因此 V4 先把 heads 分组，每组投到较低维中间表示，再合并投影到最终 attention output。这是长上下文模型里常见的“不要让输出投影吃掉注意力节省”的工程细节。

5.3 HCA 的细节：更粗的全局记忆

HCA 与 CSA 的压缩公式类似，但压缩率 m' 远大于 m ，官方配置中 $m=4$ 、 $m'=128$ 。HCA 不做 top-k sparse selection，而是在 heavily compressed entries 上做 dense attention。直觉上，CSA 解决“哪些长程片段值得精读”，HCA 解决“全局背景至少不能完全丢失”。在 1M 上下文场景，模型必须同时面对大量无关信息和少量关键证据；CSA 与 HCA 的组合正好对应这两种需求。

5.4 保真机制：SWA、RoPE、RMSNorm、Attention Sink

压缩会带来信息损失，V4 因此补了四个机制。第一，sliding window attention 分支保留最近 $n_{win}=128$ 个未压缩 KV，以弥补压缩块内无法访问同块 token 的因果限制。第二，query 与 compressed KV 在 core attention 前做 RMSNorm，抑制 attention logits 爆炸。第三，只在最后 64 个维度使用 RoPE，并在 attention output 上施加位置反向处理，使输出携带相对位置信息。第四，attention sink 给每个 head 一个可学习 sink logit，使总注意力质量可以小于 1，允许 query 在没有合适历史时不被迫分配全部注意力。

5.5 效率数字如何理解

官方报告给出了三个层级的效率比较。第一，相比 BF16 GQA8、head dimension 128 这类常见 baseline，V4 在 1M context 下 KV cache 可以压到约 2%。第二，相比已经高效的 V3.2，V4-Pro 在 1M context 下约为 27% 单 token FLOPs 与 10% KV cache；V4-Flash 约为 10% FLOPs 与 7% KV cache。第三，V4 还在 KV 存储中使用 RoPE 维度 BF16、其余维度 FP8，indexer attention 使用 FP4，并在 post-training 中对 MoE expert weights 与 indexer QK path 做 FP4 QAT。

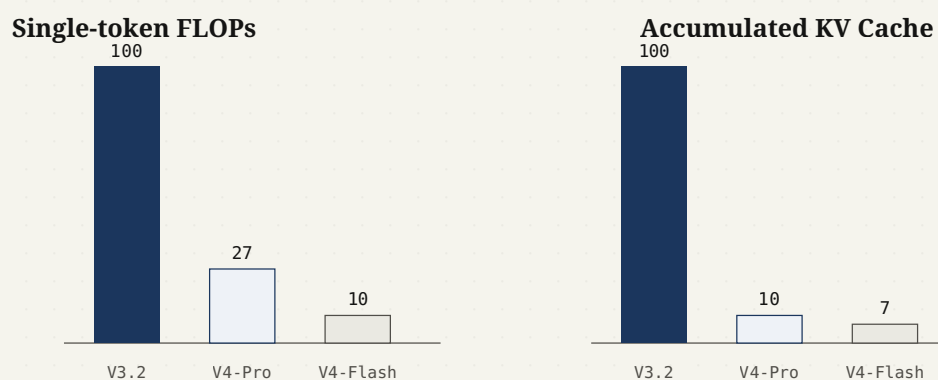
FIGURE 4 1M context efficiency, V3.2 = 100

图 4: 官方报告给出的 1M context 相对效率。数值为相对 V3.2 的比例, V3.2 设为 100。

| mHC 与 Muon: 稳定性、收敛与深层信号传播

如果只看注意力，V4 像是一个长上下文效率模型；但 mHC 与 Muon 说明 DeepSeek 同时在处理更基础的问题：当模型更深、更稀疏、连接更复杂时，训练信号如何稳定传播，优化器如何在超大矩阵上保持可控更新。

6.1 mHC: 把 residual stream 扩展成受约束混合空间

传统 residual connection 可以写成 $x_{l+1} = x_l + F_l(x_l)$ 。Hyper-Connections 的想法是把 residual stream 从一个向量扩展成 n_{hc} 个 residual channels，即 X_l 属于 $\mathbb{R}^{n_{hc} \times d}$ 。每层通过 A_l 把多通道残差混成实际 layer input，通过 C_l 把 layer output 写回多通道残差，再通过 B_l 对 residual channels 做变换。

问题是，朴素 HC 在深层堆叠时可能数值不稳定。V4 引入 mHC，把 B_l 约束到 doubly stochastic matrices 组成的 Birkhoff polytope。这样的矩阵行和、列和都为 1，元素非负，谱范数被界定在 1 以内，因而 residual transformation 是 non-expansive。更重要的是，这个集合对矩阵乘法封闭，深层堆叠时稳定性不会轻易破坏。

实现上，V4 先由输入动态生成 A、B、C 的 raw parameters，再加静态 bias 和小初始化 gate。A 与 C 通过 sigmoid 保持非负和有界；B 先指数化为正矩阵，再用 Sinkhorn-Knopp 做 20 次行列归一化，投影到近似 doubly stochastic manifold。直观地说，mHC 不是让每层任意改写 residual channels，而是要求每层以“守恒式混合”传播信息。

mHC 的价值不在于增加大量计算，而在于提供新的 scaling axis: residual width n_{hc} 可以扩展，而 inner layer 的 hidden dimension d 不必同步扩大。V4 配置中 $n_{hc}=4$ ，官方工程优化后 mHC wall-time overhead 约为 overlapped 1F1B pipeline stage 的 6.7%。

6.2 Muon: 对隐藏层矩阵做近似正交化更新

V4 把大部分模块从 AdamW 改为 Muon。Muon 的核心是对梯度动量矩阵做近似 orthogonalization，再作为更新方向。官方算法中，先计算梯度 G_t ，更新 momentum buffer M_t ，然后对 $\mu M_t + G_t$ 做 hybrid Newton-Schulz 迭代，得到近似正交化更新 O'_t ，再按矩阵维度和 rescaling factor 重新缩放 RMS，最后执行 weight decay 与参数更新。

V4 的 Muon 并非覆盖所有参数。Embedding、prediction head、mHC 的 static biases 与 gates、所有 RMSNorm weights 仍使用 AdamW；其余大部分模块使用 Muon。这个划分很合理：Muon 主要适合二维矩阵参数；norm、bias、embedding/head 等参数结构或训练敏感性不同，保留 AdamW 更保守。

hybrid Newton-Schulz 共 10 步, 前 8 步使用一组激进系数快速把奇异值推近 1, 后 2 步改用更稳定系数精确收敛。V4 还特别说明没有使用 QK-Clip, 因为 CSA/HCA 在 attention queries 与 KV entries 上直接使用 RMSNorm, 已经能抑制 attention logits 爆炸。

6.3 训练稳定性: Anticipatory Routing 与 SwiGLU Clamping

官方报告承认 V4 训练 trillion-parameter MoE 时遇到明显 instability, loss spikes 与 MoE 层 outliers 和 routing 机制有关。两个实用技巧解决了这个问题。第一是 Anticipatory Routing: 在 step t 使用当前参数计算特征, 但 routing indices 来自历史参数 $\theta_{t-\Delta t}$ 。工程上提前取 step t 的数据, 在 $t-\Delta t$ 时预计算并缓存 routing indices, loss spike 触发时短暂 rollback 并启用该模式, 稳定后恢复标准训练。

第二是 SwiGLU Clamping: 对 SwiGLU 的线性分量限制在 $[-10, 10]$, gate 分量上界限制为 10, 用于消除 outliers。这里的教训是, MoE scaling 不只是容量问题, 也是“路由反馈循环”的控制问题。路由网络把 token 分配到专家, 专家 outlier 又会反过来改变下次路由分布; 如果不切断或缓和这个循环, 模型可能在局部异常上自我放大。

6.4 这两个模块的共同意义

mHC 与 Muon 分别作用在“网络拓扑”和“优化更新”两个层面。mHC 限制层间混合的谱行为, 使 deep stack 的信息传播更稳定; Muon 约束大矩阵更新的方向, 使训练更快、更稳。再配合 Anticipatory Routing 和 SwiGLU Clamping, V4 表明大模型架构创新已经进入一个阶段: 不能只设计更稀疏的 attention, 还必须把残差、优化器、路由动态和 kernel 实现作为一个整体处理。

| 系统工程：训练、推理、KV Cache 与确定性

V4 的系统章节非常关键，因为 CSA/HCA 让 KV cache 变成异构结构，mHC 与 Muon 改变训练内存和通信模式，百万上下文又放大了 rollout、prefix reuse、确定性和沙箱执行成本。没有这些工程，算法不会变成可用模型。

7.1 MegaMoE: 把 MoE 通信隐藏在计算后面

MoE 的瓶颈通常是 expert parallelism 带来的 all-to-all 通信。V4 把每个 MoE 层拆成 Dispatch、Linear-1、Activation、Linear-2、Combine 等阶段，并设计 fine-grained expert waves。每个 wave 的专家一旦通信完成就立即计算，而不是等所有专家都到齐。稳态中，当前 wave 计算、下一 wave token transfer、已完成专家结果返回同时进行。

官方报告称，CUDA mega-kernel MegaMoE 在 NVIDIA GPU 与华为 Ascend NPU 平台都做了验证；相比强 non-fused baseline，通用推理场景加速 1.50 到 1.73 倍，RL rollout 和高速 agent serving 等 latency-sensitive 场景最高 1.96 倍。它还给硬件厂商提出了 compute/communication ratio、power headroom、低延迟通信 primitive 和低成本激活函数等建议。

7.2 TileLang 与确定性 kernel

V4 的架构如果用大量细粒度 Torch ATen operators 拼接，会带来调用开销和优化困难。DeepSeek 使用 TileLang 开发 fused kernels，并强调 Host Codegen 把 Python 侧 validation、argument marshaling 等逻辑移到生成的 host launcher 中，使每次调用的 CPU validation overhead 从几十或几百微秒降到小于 1 微秒。

另一个重点是 bitwise reproducibility。V4 为 attention、matrix multiplication、attention backward、MoE backward、mHC 小矩阵乘法等场景设计 batch-invariant 与 deterministic kernels。这样做不是形式主义，而是大规模训练调试的必要条件：当 loss spike 或后训练行为异常时，如果同一个 token 在不同 batch 位置产生不同 bitwise 输出，定位问题会变得近乎不可控。

7.3 训练框架：Muon、mHC、Contextual Parallelism

Muon 与 ZeRO 传统做法存在冲突，因为 Muon 需要完整梯度矩阵，而 ZeRO 喜欢把参数矩阵切分到多 rank。V4 对 dense parameters 限制 ZeRO 并行最大规模，用 knapsack 给 rank 分桶；超出部分用冗余 Muon update 换内存降低。对 MoE parameters，则按 expert 独立优化，flatten 同类矩阵并避免切开逻辑独立矩阵。

CSA/HCA 的 Context Parallelism 也需要重新设计。传统 CP 按序列维切分，每个 rank 持有连续 s 个 token；但压缩每 m 或 m' 个 token 才能产生 compressed KV，边界处可能跨 rank。V4 用两阶段通信：先把每个 rank 的最后 m 个未压缩 KV 发给下一个 rank，以便跨边界压缩；再 all-gather 本地 compressed KV，用 fused select-and-pad 统一整理可见范围。

7.4 推理框架: 异构 KV Cache 与 on-disk prefix reuse

V4 的 KV cache 不再是每层同构、每 token 一个固定 block。它至少包含 CSA compressed KV、HCA compressed KV、CSA indexer KV、sliding window uncompressed KV, 以及尚未凑满压缩块的 tail hidden states。传统 PagedAttention 假设很难直接覆盖这种结构, V4 因此把 cache 分成 classical KV cache 和 state cache: state cache 存 SWA 与未压缩 tail; classical KV cache 存 CSA/HCA 的 compressed entries。

on-disk KV cache storage 解决 shared-prefix 请求重复 prefilling 的问题。CSA/HCA 的 compressed KV 可以直接落盘并在 prefix hit 时读取; SWA KV 因为未压缩且体积约为 compressed CSA/HCA 的 8 倍, V4 提供三种策略: Full SWA Caching 完全存储但写入重; Periodic Checkpointing 每 p token 存一次最近 n_{win} 状态; Zero SWA Caching 不存 SWA, 命中时利用 CSA/HCA cache 重新计算最近 $n_{win} \times L$ tokens 恢复状态。

系统问题	V4 方案	为什么重要
MoE all-to-all 通信	fine-grained expert waves + fused MegaMoE kernel	让通信被计算隐藏, 尤其改善小 batch rollout。
复杂 kernel 开发	TileLang、Host Codegen、Z3 整数分析	在快速迭代 CSA/HCA 等新算子时保持性能。
可复现性	batch-invariant 与 deterministic kernels	大规模训练与 post-training 调试需要 bitwise 对齐。
长上下文并行	两阶段 contextual parallelism	解决压缩块跨 CP rank 边界的问题。
异构 cache	state cache + classical KV cache	让 CSA/HCA/SWA 不同 cache policy 共存。
shared prefix	on-disk KV cache storage	避免百万上下文重复 prefill, 提升服务效率。

预训练：数据、模型配置与训练稳定性

V4 预训练不是简单扩大 token 数。它在 V3 数据管线之上强调更长有效上下文、更高质量长文档、更大多语语料和 agentic coding 数据，并把训练长度从 4K 逐步扩到 1M。

8.1 数据构造

官方报告称，V4 预训练语料超过 32T tokens，包含数学、代码、网页、长文档和其他高质量类别。相比 V3，V4 继续过滤批量自动生成和模板化网页内容，以降低 model collapse 风险；数学和编程语料仍是核心；多语语料更大，以覆盖不同文化下的长尾知识；长文档数据被特别强调，尤其是科学论文、技术报告等具有独特学术价值的材料。

tokenizer 基本继承 V3，在其基础上增加少量用于 context construction 的 special tokens，词表仍为 128K。V4 也继承 token-splitting 与 Fill-in-Middle 策略，并使用 sample-level attention masking。对长上下文训练而言，packing 策略尤其重要：如果长文档频繁被截断，模型看到的是碎片而不是长程结构；V4 引用 fewer truncations 相关思路，尽量把不同来源文档打包到合适序列中。

8.2 训练日程

项目	V4-Flash	V4-Pro
预训练 tokens	32T	33T
最大 batch tokens	75.5M	94.4M
Peak LR	2.7e-4	2.0e-4
End LR	2.7e-5	2.0e-5
Warmup	前 2000 steps	策略类似 Flash
序列长度	4K → 16K → 64K → 1M	4K → 16K → 64K → 1M
稀疏注意力引入	前 1T dense warmup; 64K 时引入 sparse	dense stage 更长; 之后同样两阶段引入 sparse
MTP loss weight	大部分训练 0.3, LR decay 后 0.1	同 Flash
Load balancing bias speed	0.001	0.001
Sequence-wise balance loss	0.0001	0.0001

8.3 Base model 评测透露了什么

V4 报告 Table 1 对比了 V3.2-Base、V4-Flash-Base 和 V4-Pro-Base。最值得注意的是，V4-Flash-Base 虽然每 token 激活参数从 V3.2 的 37B 降到 13B，总参数从 671B 降到 284B，但在多数 benchmark 上超过 V3.2-Base。这说明 V4 的架构、数据和训练改进不仅提升长上下文效率，也提升了参数效率。

V4-Pro-Base 则基本全面领先，尤其在知识密集与长上下文任务上提升显著：SimpleQA verified 从 V3.2-Base 的 28.3 到 V4-Pro-Base 的 55.2；FACTS Parametric 从 27.1 到 62.6；LongBench-V2 从 40.2 到 51.5。代码与数学并非每项都单调提升，例如 BigCodeBench 上 V3.2-Base 63.9 高于 V4-

Pro-Base 59.2，但 HumanEval、GSM8K、MATH 等项 V4-Pro-Base 更强。更专业的结论是：V4-Pro-Base 是更强通用基础模型，但单项 benchmark 的增减仍受数据、评测和任务形态影响。

指标	V3.2-Base	V4-Flash-Base	V4-Pro-Base
Activated Params	37B	13B	49B
Total Params	671B	284B	1.6T
MMLU-Pro	65.5	68.3	73.5
SimpleQA verified	28.3	30.1	55.2
FACTS Parametric	27.1	33.9	62.6
HumanEval	62.8	69.5	76.8
MATH	60.5	57.4	64.5
LongBench-V2	40.2	44.7	51.5

8.4 训练稳定性的实际含义

V4 报告没有把 loss spike 一笔带过，而是明确提出 Anticipatory Routing 与 SwiGLU Clamping。这意味着 V4 的架构复杂度已经接近一个临界点：MoE、hybrid attention、mHC、Muon 和超长序列训练相互耦合，传统“rollback 后继续跑”的办法不足以保证大规模稳定。未来如果要复现类似架构，训练监控、路由状态、激活 outlier、专家负载和确定性 kernel 都必须纳入第一等工程目标。

| 后训练：专家、GRM、OPD、工具调用与沙箱

V4 后训练的核心不是“再做一次 RL”，而是先让多个领域专家充分分化，再通过 full-vocabulary On-Policy Distillation 把能力合并到统一模型。这是从单一 policy optimization 转向多教师能力整合。

9.1 三种 reasoning modes

V4 支持 Non-think、Think High、Think Max 三种模式。Non-think 面向快速日常任务；Think High 面向复杂问题、计划和中等风险决策；Think Max 用最大 reasoning effort 推到能力边界。训练上，不同模式对应不同 RL length penalty 与 context window，因此输出推理 token 长度不同。格式上，思考内容由 <think> 和 </think> 标记；Think Max 还在 system prompt 前加入特殊指令，要求更彻底的问题分解和逻辑压力测试。

9.2 Specialist Training 与 Generative Reward Model

V4 先针对数学、代码、Agent、指令跟随等目标领域训练独立专家。每个专家先用高质量领域数据 SFT，再用 GRPO 进行领域 RL。易验证任务使用规则 verifier 或测试用例；难验证任务不依赖传统 scalar reward model，而是使用 rubric-guided RL data 与 Generative Reward Model。更准确地说，actor network 自身承担 GRM 角色，把评价能力和生成能力联合优化。

这个设计与 R1 的关系很清楚：R1 证明结果奖励可以诱导推理；V4 则把这个思想推广到难验证任务，用生成式判断取代单个标量分数。它的潜在优势是模型可以用自己的推理过程进行评判，减少人类标注量；风险是 GRM 仍可能受到 reward hacking 或自我偏见影响，需通过 rubrics、验证和人评约束。

9.3 工具调用：DSML 与 XML schema

V4 引入新的 tool-call schema，使用特殊 |DSML| token 和 XML 风格调用格式。核心目的是降低 escaping failure 与工具调用错误。JSON 在嵌套字符串、代码、引号和多工具场景中容易出错；XML 风格虽然更冗长，但边界更清晰，尤其适合模型输出结构化工具调用。

V4 还改进了 interleaved thinking 的上下文管理。V3.2 在 tool-result rounds 中保留 reasoning trace，但新用户消息到来时丢弃。V4 在真正的 tool-calling scenarios 中跨全部对话保留 reasoning content，包括跨用户消息边界；普通聊天场景则仍丢弃旧 reasoning content，以避免无谓上下文膨胀。这个变化只有在 1M context 下才真正合理，因为长期 Agent 任务需要累积状态，而不是每轮重建推理脉络。

9.4 Quick Instruction：不用小模型也能做前置任务

聊天系统常需要先判断是否搜索、生成检索 query、判断权威性、识别领域、提取 URL 等。传统做法是另起小模型，但会重复 prefill，无法复用大模型 KV cache。V4 引入 Quick Instruction：在输入后追加特殊 token，例如 <|action|>、<|query|>、<|authority|>、<|domain|>，让同一个模型利用已经计算好

的 KV cache 并行执行这些辅助任务。它本质上是把 router / classifier / query generator 等系统小功能内生到主模型里。

9.5 OPD: 多专家能力合并的关键

V4 把 V3.2 的 mixed RL stage 替换为 On-Policy Distillation。给定多个专家模型，统一学生模型在自己的生成轨迹上学习教师分布，目标是 reverse KL。与离线蒸馏不同，OPD 是 on-policy：训练样本来自 student 自己当前策略的轨迹，因此更能纠正 student 实际会访问的状态分布。

官方报告特别强调 full-vocabulary logit distillation，而不是只在采样 token 上做 KL 估计。后者更省资源，但梯度方差大、稳定性差。full-vocabulary KL 更忠实、更稳定，但代价巨大，因为词表超过 100K、教师超过 10 个、模型可达 trillion scale。V4 的工程解法是：教师权重集中 offload，按需加载；forward pass 只缓存 last-layer hidden states，不显式物化全部 logits；训练时再通过对应 prediction head 重建 logits，并用 TileLang kernel 计算精确 KL。

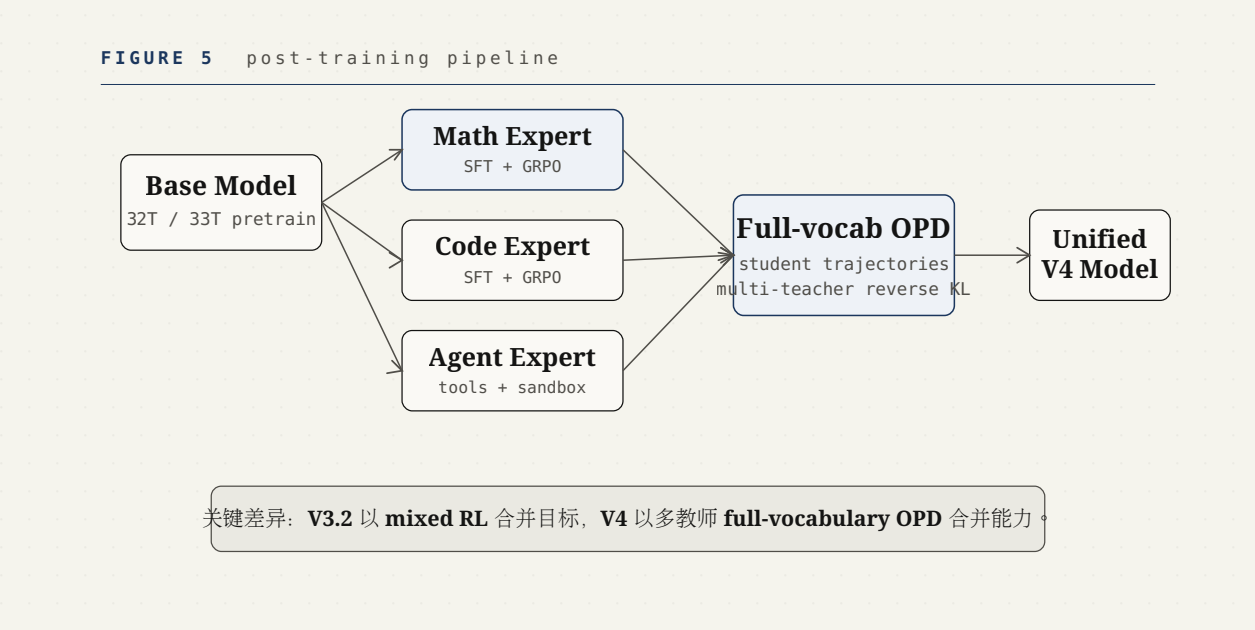


图 5: V4 后训练先分化专家，再通过 OPD 合并到统一学生模型。

9.6 FP4 QAT、rollout WAL 与 DSec

V4 在 post-training 中引入 FP4 quantization-aware training。量化对象包括 MoE expert weights 和 CSA indexer 的 QK path。对 expert weights，FP32 master weights 先量化到 FP4，再无损 dequantize 到 FP8 参与计算；反向传播通过 STE 把梯度传回 FP32 master weights。对 indexer，QK activations 以 FP4 cache、load 和 multiply，index scores 从 FP32 量化到 BF16，据称 top-k selector 加速 2 倍且 KV entry recall 保持 99.7%。

长上下文 RL/OPD 需要大量 rollout。V4 的 rollout service 支持 preemptible 与 fault-tolerant：每生成一个 token，就把它写入 token-granular Write-Ahead Log。任务被抢占时保存未完成请求 KV cache；恢复时用 WAL 与 KV cache 继续 decoding。官方特别指出，若遇到中断就从头重生成，会引入 length bias，因为短响应更容易在中断前完成并“幸存”。

Agent 后训练和评测还需要沙箱。DeepSeek Elastic Compute 使用 Rust 组件 Apiserver、Edge、Watcher，构建在 3FS 分布式文件系统之上，支持 Function Call、Container、microVM、fullVM 四种执行 substrate。它的目标是让大规模异构 Agent workload 在安全、可复现、可抢占、可恢复的环境中执行。

评测对比：能力、效率、成本与边界

V4 的评测应分三层看：基础模型能力证明架构有效，post-trained model 对比说明开放模型的前沿位置，真实任务评估揭示产品体验。所有这些数据主要来自 DeepSeek 官方内部评测，结论应带着来源边界阅读。

10.1 V4-Pro-Max 与其他模型

官方 Table 6 将 DeepSeek-V4-Pro-Max 与 Claude Opus 4.6 Max、GPT-5.4 xHigh、Gemini-3.1-Pro High、Kimi K2.6 Thinking、GLM-5.1 Thinking 比较。整体上，V4-Pro-Max 在开放模型中非常强：SimpleQA-Verified 57.9，高于 K2.6 的 36.9 与 GLM-5.1 的 38.1；Codeforces rating 3206，高于 GPT-5.4 的 3168 和 Gemini-3.1-Pro 的 3052；Apex Shortlist 90.2，高于表中其他模型。它在 GPQA、HLE、Apex 等高难知识/推理项上仍落后于 Gemini-3.1-Pro 或 GPT-5.4。

Agent 项更复杂。SWE Verified 上 V4-Pro-Max 为 80.6，与 Gemini-3.1-Pro 80.6、Opus 4.6 80.8 基本齐平；Terminal Bench 2.0 为 67.9，低于 GPT-5.4 的 75.1，但高于 Opus 4.6 的 65.4；MCPAtlas Public 73.6，与 Opus 4.6 的 73.8 接近；Toolathlon 51.8，低于 GPT-5.4 的 54.6，高于 Opus 4.6 和 Gemini-3.1-Pro。

Benchmark	Opus-4.6 Max	GPT-5.4 xHigh	Gemini-3.1 High	K2.6	GLM-5.1	DS-V4-Pro Max
SimpleQA-Verified	46.2	45.3	75.6	36.9	38.1	57.9
GPQA Diamond	91.3	93.0	94.3	90.5	86.2	90.1
HLE	40.0	39.8	44.4	36.4	34.7	37.7
LiveCodeBench	88.8	-	91.7	89.6	-	93.5
Codeforces Rating	-	3168	3052	-	-	3206
Apex Shortlist	85.9	78.1	89.1	75.5	72.4	90.2
Terminal Bench 2.0	65.4	75.1	68.5	66.7	63.5	67.9
SWE Verified	80.8	-	80.6	80.2	-	80.6
MCPAtlas Public	73.8	67.2	69.2	66.6	71.8	73.6
Toolathlon	47.2	54.6	48.8	50.0	40.7	51.8

10.2 Reasoning effort 的作用

V4 把推理模式显式产品化。Table 7 显示，从 Non-think 到 High，再到 Max，许多任务显著提升。V4-Pro 在 HLE 上从 7.7 到 34.5，再到 37.7；LiveCodeBench 从 56.8 到 89.8，再到 93.5；Apex Shortlist 从 9.2 到 85.5，再到 90.2。V4-Flash 也类似：HMMT 从 40.8 到 91.9，再到 94.8；IMOAnswerBench 从 41.9 到 85.1，再到 88.4。

这说明 V4 的能力不能只用一个模型名表示，而要同时说明 reasoning mode 和 token budget。对用户而言，Non-think 是低延迟模式，High 是日常复杂任务的均衡模式，Max 是高风险、高难、可接受慢响应任务的能力上限模式。对研究者而言，这表明 test-time scaling 已经从“提示模型多想一会儿”变成训练、格式、窗口和服务策略共同定义的系统参数。

10.3 1M 上下文：合成 retrieval 与真实 CorpusQA

V4 的 1M context 评测包含 OpenAI MRCR 与 CorpusQA。MRCR 是多 needle retrieval 型任务，更强调在长上下文中找证据；CorpusQA 更接近真实语料级分析。官方报告称 V4-Pro 在 MRCR 上超过 Gemini-3.1-Pro，但低于 Claude Opus 4.6；在 CorpusQA 上也优于 Gemini-3.1-Pro。图 9 显示，MRCR 在 128K 内相对稳定，256K 后开始下降，但 1M 仍保留明显检索能力。

这里需要专业地看边界。1M context 不等于“模型能无损理解 1M token”。它表示模型能在这个窗口内进行有效访问，并且成本与 cache 可承受。实际任务仍会受信息密度、证据位置、干扰项、prompt 组织、工具调用和推理预算影响。V4 的贡献在于把 1M 从不可承受的显存/算力问题变成可工程调度的问题。

10.4 真实任务：中文写作、搜索、白领任务、代码 Agent

官方真实任务评估很有价值，因为它揭示 benchmark 以外的产品特性。中文 functional writing 中，V4-Pro 相比 Gemini-3.1-Pro 总体 win rate 为 62.65% 对 34.10%；creative writing 的 writing quality win rate 为 77.48%。但在复杂指令跟随和多轮写作上，V4-Pro 相比 Claude Opus 4.5 总体为 45.9% 对 52.0%，说明在极复杂约束和多轮场景仍有差距。

搜索方面，agentic search 相比 RAG 总体 win rate 为 61.7% 对 18.3%，但平均 tool calls 为 16.2，prefill tokens 13,649，output tokens 1,526，成本只比 RAG 略高但工具调用大多可并行。白领任务中，V4-Pro-Max 对 Opus-4.6-Max 的整体 non-loss rate 为 63%，优势主要在任务完成与内容质量，短板在 instruction following 和幻灯片视觉设计。内部 R&D coding benchmark 上，V4-Pro-Max pass rate 67%，接近 Opus 4.5 的 70%，低于 Opus 4.6 Thinking 的 80%。

10.5 如何做公平比较

比较 V4 与其他模型时必须同时控制五件事：模型大小和开放性、reasoning effort、context length、工具协议、评测 harness。比如某个 Agent benchmark 的分数可能受工具 schema、上下文管理、是否保留 thinking、搜索 API 和 sandbox 环境影响；某个长上下文分数可能受 prompt packing 与证据位置影响；某个代码分数可能受候选采样数、测试集构造和工具步数影响。本文采用官方数据作为技术分析依据，但不把它们视为独立第三方排名。

创新点分层、局限与实践判断

V4 的创新可以分成四层：模型结构、优化训练、系统基础设施、后训练产品化。它的强项是把这四层真正绑在一起；它的局限也来自复杂度本身。

11.1 创新点一：压缩注意力替代单纯稀疏选择

CSA/HCA 是 V4 最核心的结构创新。它不是把 attention 稀疏化这么简单，而是先改变 KV 表示的时间分辨率。CSA 以 $m=4$ 的较小压缩率保持较高分辨率，再通过 top-k 选择控制成本；HCA 以 $m'=128$ 的大压缩率提供粗粒度全局记忆。这个设计比单纯 DSA 更适合 1M context，因为它把“历史太长”变成“多分辨率历史摘要”。

11.2 创新点二：mHC 为深层 MoE 提供新的连接维度

mHC 的价值在于让 residual path 本身可学习、可扩展、可稳定。它不像普通 residual 那样只是一条 identity shortcut，也不像任意 HC 那样可能在深层放大信号，而是通过 Birkhoff polytope 约束保持非扩张。对长期发展而言，这可能比单次 benchmark 提升更重要，因为它打开了“扩展 residual topology”这条架构路线。

11.3 创新点三：Muon 与稳定技巧进入 trillion-scale 主线

Muon 在 V4 中不是实验小技巧，而是用于大多数模块的主优化器。它将矩阵更新近似正交化，配合 hybrid Newton-Schulz、BF16 迭代、ZeRO bucket 策略和 MoE expert flattening。再加上 Anticipatory Routing 和 SwiGLU Clamping，V4 把优化器、路由稳定和 outlier 控制放到了核心位置。

11.4 创新点四：OPD 把后训练从“混合目标”转向“专家合并”

V3.2 的 mixed RL 把 reasoning、agent、alignment 混在一个 RL stage 中。V4 改为先训练多个专家，再用 OPD 合并。这个变化值得重视，因为大模型能力越来越像多个分布的集合：数学、代码、搜索、写作、Agent、工具使用、格式控制各自有不同 reward 与数据形态。OPD 给出一种工程上可扩展的能力合并路线。

11.5 局限：复杂、未完全解释、依赖系统

官方结论也承认 V4 为追求极致长上下文效率，保留了许多初步验证有效的组件和 tricks，使架构较复杂。Anticipatory Routing 与 SwiGLU Clamping 有效，但底层原理仍未充分理解。换句话说，V4 是非常强的工程系统，但还不是“优雅最小解”。未来方向包括把架构蒸馏到更本质的设计、研究训练稳定性原理、探索新的稀疏维度、降低长上下文延迟、增强长程多轮 Agent、加入多模态能力。

11.6 实践判断: 什么时候 V4 路线最有价值

使用场景	V4 路线价值	注意事项
百万级文档分析	CSA/HCA 与 on-disk KV 能显著降低长上下文成本	仍需良好文档组织, 1M 不等于无损理解。
长程 Agent workflow	跨工具回合保留 thinking, 可减少重复重建状态	只有真正 tool-calling path 才能完全受益。
高难数学/代码推理	Max mode 与 test-time scaling 带来明显提升	响应更慢, token 成本更高, 需按任务风险选择。
企业知识搜索	Quick Instruction 与 agentic search 降低前置任务和检索迭代成本	需要控制工具质量、权限、引用和缓存策略。
自部署开放权重	MIT 权重许可降低研究与部署门槛	1.6T Pro 的部署要求极高; Flash 更现实。

最终判断

DeepSeek V4 的真正意义是把开放模型竞争从“谁的 benchmark 更高”推进到“谁能把长上下文、推理、工具、训练稳定和服务成本打通”。它不是一个纯算法论文，而是一份架构和基础设施联合演进的工程宣言。

术语表与参考来源

12.1 关键术语

术语	解释
MoE	Mixture-of-Experts。每个 token 只激活部分专家，用较低计算成本获得较大总参数容量。
DeepSeekMoE	DeepSeek 系列使用的 MoE 架构，包含 shared experts、fine-grained routed experts 与 load balancing 设计。
MLA	Multi-head Latent Attention。V3 使用的高效注意力形式，通过 latent KV 降低 cache。
DSA	DeepSeek Sparse Attention。V3.2 引入的稀疏注意力，用 lightning indexer 选择 top-k KV entries。
CSA	Compressed Sparse Attention。V4 引入，先压缩 KV，再对 compressed entries 做 top-k sparse attention。
HCA	Heavily Compressed Attention。V4 引入，以更大压缩率压缩 KV，并在压缩条目上保留 dense attention。
mHC	Manifold-Constrained Hyper-Connections。把 residual stream 扩展为多通道，并把 residual mixing matrix 约束到 doubly stochastic manifold。
Muon	基于矩阵更新近似正交化的优化器。V4 对多数模块使用 Muon，对 embedding、head、RMSNorm 等保留 AdamW。
GRPO	Group Relative Policy Optimization。R1 与 V4 specialist RL 中使用的强化学习算法，通过一组 samples 的相对奖励估计 advantage。
OPD	On-Policy Distillation。学生在自己的生成轨迹上学习多个教师模型分布，V4 用它合并领域专家能力。
FP4 QAT	FP4 Quantization-Aware Training。训练阶段显式适配 FP4 量化误差，使部署时可用更低精度加速和省显存。
DSec	DeepSeek Elastic Compute。用于 Agent 后训练和评测的沙箱平台，支持 Function Call、Container、microVM、fullVM。

12.2 参考来源

- DeepSeek 官方透明中心：DeepSeek-V4 条目，Release Date 2026-04-24。
- DeepSeek-AI, DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence, Hugging Face PDF, 2026。
- DeepSeek-AI, DeepSeek V4 Model Card, 2026。
- DeepSeek-AI, DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models, 2025。
- DeepSeek-AI, DeepSeek-V3 Technical Report, arXiv:2412.19437, 2024/2025。
- DeepSeek-AI, DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning, arXiv:2501.12948, 2026 version。

12.3 边界说明

本文是技术解读，不是 DeepSeek 官方文档。所有 benchmark、模型规模、训练配置和发布时间以官方公开材料为依据；涉及“为什么这样设计”“该如何理解”的部分为本文基于报告内容做出的工程推断。若后续 DeepSeek 更新 V4 技术报告、模型卡或权重仓库，本文结论需按新材料复核。